

Day 2 — 传感器与 OLED 显示驱动（完整代码版 • F407VET6 新核心板版）

已按新核心板更新：三传感器共软 I2C 总线1(PB8/PB9)、OLED 独立总线2(PD6/PD7)、QMI8658 地址 0x6A；代码以工程 `pro_mqtt_cloud` 的 BSP 封装为准

今日目标： OLED 屏上同时显示温度、振动幅度、电流三组数值，手摸/晃动有变化，连续运行 10 分钟不死机。
开发顺序： 接线与总线分配 → 总线扫描 → SHT30 → QMI8658 → INA226 → OLED → 整合显示。逐个调通，不要一次全接。

1. 第一步：硬件接线与总线分配（新板版）

新核心板为 **STM32F407VET6 (LQFP100)**，I2C 外设全部用**软件模拟 I2C**（GPIO 时序），两条独立总线：

总线	引脚	挂载器件	7 位地址	说明
软 I2C 总线1	PB8 = SCL, PB9 = SDA	SHT30 / QMI8658 / INA226	0x44 / 0x6A / 0x40	三个传感器共用，SCL/SDA 各加 4.7kΩ 上拉到 3.3V
软 I2C 总线2	PD6 = SCL, PD7 = SDA	SSD1306 OLED	0x3C	OLED 独立总线（模块自带 I2C 上拉）

与旧板差异：旧方案把 OLED 也挂 I2C1、INA226 挂独立 I2C2(PB10/PB11)。新板为避免与 ESP8266 的 USART3(PB10/PB11) 冲突，**INA226 并入总线1、OLED 拆到总线2**——这是本新板版的推荐做法，无引脚冲突。

2. 第二步：总线扫描（先确认硬件通了）

工程已封装软件 I2C 主机驱动 `bsp_i2c_soft.c`（PB8/PB9）。在 `APP_Init()` 中先 `BSP_I2C_Soft_Init()`，再用 `BSP_I2C_Soft_Probe()` 逐个探测器件：

```
/* 总线1 探测：SHT30 / QMI8658 / INA226 */
if (BSP_I2C_Soft_Probe(0x44) == 0) BSP_UART1_Printf("SHT30   found @0x44\n");
if (BSP_I2C_Soft_Probe(0x6A) == 0) BSP_UART1_Printf("QMI8658 found @0x6A\n");
if (BSP_I2C_Soft_Probe(0x40) == 0) BSP_UART1_Printf("INA226  found @0x40\n");
```

```
");  
/* 总线2 (OLED) 为独立 GPIO 时序, 直接调 BSP_OLED_Init() 后点亮即可,  
* 若能正常点亮/显示即说明 0x3C 通信正常 */
```

预期结果：总线1 应扫描到 0x44、0x6A、0x40；总线2 的 OLED(0x3C) 能正常点亮显示。

💡 扫描不到设备的排查顺序：(1) 检查 VCC/GND 是否接对；(2) SDA/SCL 是否接反；(3) 总线1 上拉电阻是否加 (4.7kΩ 到 3.3V) ；(4) 逻辑分析仪/示波器看 SCL 有无时钟；(5) 换一根杜邦线。

3. 第三步：SHT30 温湿度驱动

工程封装：bsp_sht30.c (软 I2C 总线1, 7 位地址 **0x44**, ADDR 引脚接 GND) 。

```
/* bsp_sht30.h 核心 API */  
#define SHT30_I2C_ADDR 0x44U  
uint8_t BSP_SHT30_Init(void); /* 探测 + 软件复位 + 清状态 */  
uint8_t BSP_SHT30_Read(BSP_SHT30_Data_t *data); /* 单次测量, 阻塞约20ms */  
  
/* 裸机测试 (main.c USER CODE 2 之后) */  
BSP_SHT30_Data_t sht;  
if (BSP_SHT30_Init() == 0) BSP_UART1_Printf("SHT30 init OK  
  
");  
while (1) {  
    if (BSP_SHT30_Read(&sht) == 0)  
        BSP_UART1_Printf("Temp: %d.%d C, Humi: %d.%d %%RH  
  
",  
                           sht.temperature_int, sht.temperature_dec, sht.humidity_int, sht.humidity_dec);  
    else  
        BSP_UART1_Printf("SHT30 read error  
  
");  
    HAL_Delay(1000);  
}
```

物理量换算 (SHT30 手册) : $\text{Temp}(^{\circ}\text{C}) = -45 + 175 \times \text{raw} / 65535$; $\text{RH}(\%) = 100 \times \text{raw} / 65535$; CRC8 多项式 0x31, 初值 0xFF。

4. 第四步：QMI8658 六轴驱动

工程封装：bsp_qmi8658.c (软 I2C 总线1, 7 位地址 **0x6A** (SAO/AD0 接 GND)) 。

QMI8658 地址说明：QMI8658A 的 7 位地址由 SAO/AD0 引脚决定：SAO=GND → **0x6A**；SAO=VDD → 0x6B。本设计**统一用 0x6A (AD0 接 GND)**，与官方寄存器手册一致，接线时务必确认模块 AD0 已接地。

```

/* bsp_qmi8658.h 核心 API（寄存器配置依据官方手册 Rev.A 考证） */
#define QMI8658_I2C_ADDR 0x6AU
uint8_t BSP_QMI8658_Init(void); /* WHO_AM_I=0x05 校验 */
uint8_t BSP_QMI8658_Read(BSP_QMI8658_Data_t *data); /* 连续读12字节原始值 */

/* 寄存器要点: CTRL1=0x40(地址自增) CTRL2=0x30(±16g) CTRL3=0x30(±128dps) CTRL7=0x03(使能) */
/* 灵敏度: accel(g)=raw/2048, gyro(dps)=raw/256 */

/* 裸机测试: 计算振动幅度(模值) */
BSP_QMI8658_Data_t qmi;
if (BSP_QMI8658_Init() == 0) BSP_UART1_Printf("QMI8658 init OK (WHO_AM_I=0x05)

");
while (1) {
    if (BSP_QMI8658_Read(&qmi) == 0) {
        float ax = qmi.ax / 2048.0f, ay = qmi.ay / 2048.0f, az = qmi.az / 2048.0f;
        float vib = sqrtf(ax*ax + ay*ay + az*az);
        BSP_UART1_Printf("ACC: %6d %6d %6d | Vib: %.3f g

", qmi.ax, qmi.ay, qmi.az, vib);
    }
    HAL_Delay(100);
}

```

QMI8658 常见坑: (1) WHO_AM_I 应为 0x05, 读出来先看实际值; (2) CTRL 寄存器位定义以手册为准, 数据全 0 或不动先查使能位; (3) 部分模块需先退出 standby。若 2 小时调不通, 换 MPU6050 (地址 0x68) 不要死磕。

5. 第五步: INA226 电流检测驱动

工程封装: `bsp_ina226.c` (软 I2C 总线1, 7 位地址 **0x40 (A0/A1 接 GND)**, 采样电阻 0.1Ω)。

```

/* bsp_ina226.h 核心 API */
#define INA226_I2C_ADDR 0x40U
uint8_t BSP_INA226_Init(void); /* 配置校准+连续模式 */
uint8_t BSP_INA226_Read(BSP_INA226_Data_t *data); /* 电压/电流/功率 */

/* 参数: Rshunt=0.1Ω, Current_LSB=0.5mA, CAL=0.00512/(0.0005*0.1)=102 */

/* 裸机测试 */
BSP_INA226_Data_t ina;
if (BSP_INA226_Init() == 0) BSP_UART1_Printf("INA226 init OK

");
while (1) {
    if (BSP_INA226_Read(&ina) == 0)
        BSP_UART1_Printf("Bus: %u mV, I: %d mA, P: %lu mW

```

```

",
    ina.bus_mv, ina.current_ma, (unsigned long)ina.power_mw);
HAL_Delay(500);
}

```

6. 第六步：OLED 显示（独立总线2 PD6/PD7）

工程封装：bsp_oled.c（SSD1306 0.96" 128×64，PD6/PD7 软件 I2C，地址 0x3C）。

```

/* bsp_oled.h 核心 API */
void BSP_OLED_Init(void);
void BSP_OLED_Clear(void);
void BSP_OLED_ShowString(uint8_t page, uint8_t col, const char *str);    /* 6x8 */
void BSP_OLED_ShowString8x16(uint8_t page, uint8_t col, const char *str); /* 8x16 */

/* 三路数据同屏（6x8 小字体，共8行） */
BSP_OLED_Init();
BSP_OLED_Clear();
BSP_OLED_ShowString(0, 0, "Motor Monitor");
BSP_OLED_ShowString(2, 0, "Temp: 26.3 C");
BSP_OLED_ShowString(4, 0, "Vib : 1.02 g");
BSP_OLED_ShowString(6, 0, "Cur : 1234 mA");

```

7. 整合显示（采集 → 显示循环）

```

while (1) {
    BSP_SHT30_Read(&sht);          /* 总线1 */
    BSP_QMI8658_Read(&qmi);        /* 总线1 */
    BSP_INA226_Read(&ina);         /* 总线1 */
    /* 组装字符串并刷新 OLED（总线2，互不干扰） */
    char line[22];
    snprintf(line, sizeof(line), "Temp:%d.%dC", sht.temperature_int, sht.temperature_dec);
    BSP_OLED_ShowString(2, 0, line);
    snprintf(line, sizeof(line), "Vib:%.2fg", vib);
    BSP_OLED_ShowString(4, 0, line);
    snprintf(line, sizeof(line), "Cur:%d mA", ina.current_ma);
    BSP_OLED_ShowString(6, 0, line);
    HAL_Delay(100);
}

```

Day 2 验收：OLED 屏上同时显示温度、振动幅度、电流三组数值，手摸 SHT30 温度上升、晃动板子振动幅度变化、接入电机负载后电流变化；连续运行 10 分钟不死机。

✦ 说明：以上为 Day 2 阶段裸机验证代码（工程 BSP 封装）。正式工程中三路采集已整合进 FreeRTOS 任务（Sht30Task/Qmi8658Task/Ina226Task，见 app_tasks.c），统一快照经 APP_SENSOR 供 OLED/MQTT 使用——接线与本页完全一致。